FIT3155: Week 7 Lab Sheet

(Questions are based on Week 6 topic dealing with semi-numerical algorithms.)

Objectives: Develop an understanding of Karatsuba big integer multiplication, modular exponentiation, and the Miller-Rabin primality test.

Conceptual exercises

1. Suppose $u = (u_1 u_2 \dots u_d)_\beta$ is a d-digit integer represented in base $\beta \geq 2$, where each digit $u_i \in \{0, 1, \dots, \beta - 1\}$. Write a formula that allows you to interpret this positional notation in base β as an integer in $\mathbb{Z}$.

   Ans: 1

   Each base-β digit u_i is interpreted according to its position (place value) in the representation. Specifically, the rightmost digit u_d is the least significant digit and contributes as a multiple of $\beta^0 = 1$. The next digit u_{d-1} contributes as a multiple of β^1, u_{d-2} as a multiple of β^2 and so on, until the leftmost digit u_1 which contributes as a multiple of β^{d-1}.

   Therefore $U = u_1\beta^{d-1} + u_2\beta^{d-2} + \cdots + u_{d-1}\beta + u_d = \sum_{i=1}^{d} \beta^{d-i} u_i$

2. Python uses arbitrary-precision integers by default. In what base are integers represented internally in Python (note: the answer is not base 2)?

   Ans: 2

   Suggested reading:
   `https://levelup.gitconnected.com/how-python-represents-integers-using-bignum-f8f0574d0d6b`.

   Note: the technical specification of how Python represents integers internally on computers is not examinable. The main intention here is that, while we are dealing with integer-based algorithms, it is useful to be aware of the under-the-hood considerations involved in arbitrary-precision integers. Moreover, in the practical implementation of Karatsuba's divide-and-conquer method (next question), this insight helps determine how to handle the base case of the recursion efficiently.

3. In the lecture, Karatsuba's algorithm was derived for multiplying two $2d$-bit integers. Generalise this decomposition to obtain a similar divide-and-conquer structure for Karatsuba's algorithm for multiplying two $2d$-digit integers in an arbitrary base b. (After this, reason about which base this algorithm would be better implemented in when done on a computer architecture with a word size of w, where multiplication of w-bit integers is supported as a single hardware operation.)

Ans: 3

Exactly as this was decomposed in base-2 in the lecture, assume we have two $2d$-digit numbers in base β:

$$u = (\underbrace{u_{2d-1}u_{2d-2}\cdots u_d}_{U_1}\underbrace{u_{d-1}u_{d-2}\cdots u_0}_{U_0})_{\text{base-}\beta} \implies u = \beta^d U_1 + U_0.$$

Similarly,

$$v = (\underbrace{v_{2d-1}v_{2d-2}\cdots v_d}_{V_1}\underbrace{v_{d-1}v_{d-2}\cdots v_0}_{V_0})_{\text{base-}\beta} \implies v = \beta^d V_1 + V_0.$$

Therefore, the product $uv = (\beta^d U_1 + U_0)(\beta^d V_1 + V_0)$ can be expanded as:

$$\begin{aligned} uv &= \beta^{2d}U_1V_1 + \beta^d(U_0V_1 + U_1V_0) + U_0V_0 \\ &= \beta^{2d}U_1V_1 + \beta^d\left(U_1V_1 - U_1V_1 + U_0V_1 + U_1V_0 - U_0V_0 + U_0V_0\right) + U_0V_0 \\ &= \beta^{2d}U_1V_1 + \beta^d\left(U_1V_1 - (U_1 - U_0)(V_1 - V_0) + U_0V_0\right) + U_0V_0 \\ &= \boxed{(\beta^{2d} + \beta^d)U_1V_1 - \beta^d(U_1 - U_0)(V_1 - V_0) + (\beta^d + 1)U_0V_0} \end{aligned}$$

The boxed formula forms the basis of divide and conquer in any base β.

About choosing an efficient base case on computers, in theory, the subproblems can be reduced until multiplication of two 1-bit integers becomes the base case (which corresponds to base $\beta = 2$ as handled in the lecture). However, this would be unnecessary and inefficient, especially when machines with some word size w with built-in hardware to support basic arithmetic operations (including multiplication).

A practical rule of thumb is to choose a base of the form $\beta = 2^k$ with $k \approx w$. The value of k is chosen so that the product of two base-β digits, together with any carry digits and, where relevant, accounting for signed integer representations fits within a machine word and can be handled efficiently on hardware. Such a choice ensures that Karatsubai's divide and conquer algorithm terminates earlier rather than recursing all the way down to 1-bit subproblems before returning.

4. Prove that the divide-and-conquer approach for integer multiplication (irrespective of the base) has time complexity of $O(d^{\log_2 3})$ by explicitly solving the following recurrence relation (without using the Master Theorem):

$$T(2d) = 3T(d) + cd,$$

where c is a constant.

Ans: 4

Let us decompose recursively the problem size by half in each step until in some iteration the subproblem $T\left(\frac{d}{2^k}\right)$ reaches the base case $T(1)$. (This implies $\frac{d}{2^k} = 1$ or in other words $k = \log_2 d$.)

$$\begin{aligned} T(2d) &= 3T(d) + cd \\ &= 3\left(3T\left(\frac{d}{2}\right) + c\frac{d}{2}\right) + cd = 3^2T\left(\frac{d}{2}\right) + cd\left(1 + \frac{3}{2}\right) \\ &= 3^2\left(3T\left(\frac{d}{4}\right) + c\frac{d}{4}\right) + cd\left(1 + \frac{3}{2}\right) = 3^3T\left(\frac{d}{4}\right) + cd\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2\right) \\ &= 3^3\left(3T\left(\frac{d}{8}\right) + c\frac{d}{8}\right) + cd\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2\right) = 3^4T\left(\frac{d}{8}\right) + cd\left(1 + \frac{3}{2} + \left(\frac{3}{2}\right)^2 + \left(\frac{3}{2}\right)^3\right) \\ &\vdots \\ &= 3^k\left(3T(\frac{d}{2^k}) + c\frac{d}{2^k}\right) + cd\sum_{i=0}^{k-1}\left(\frac{3}{2}\right)^i = \boxed{3^{k+1}\underbrace{T\left(\frac{d}{2^k}\right)}_{T(1)=b(\text{constant})} + cd\sum_{i=0}^{k}\left(\frac{3}{2}\right)^i} \end{aligned}$$

The last summation on the right-hand side, $\sum_{i=0}^{k} \left(\frac{3}{2}\right)^i$, is a geometric series sum across $k+1$ terms, where the first term is $1 = \left(\frac{3}{2}\right)^0$ and each subsequent term is obtained by multiplying previous term in the summation by $r = \frac{3}{2}$.

Therefore, $\sum_{i=0}^{k} \left(\frac{3}{2}\right)^i = \frac{\left(\frac{3}{2}\right)^{k+1}-1}{\frac{3}{2}-1} = \frac{\left(\frac{3}{2}\right)^{k+1}-1}{\frac{1}{2}} = 2\left(\left(\frac{3}{2}\right)^{k+1} - 1\right) = \left(\frac{3^{k+1}-2^{k+1}}{2^k}\right)$.

This gives

$$\begin{aligned}
T(2d) &= 3^{k+1} T\left(\frac{d}{2^k}\right) + cd\left(\frac{3^{k+1}-2^{k+1}}{2^k}\right) \\
&= 3^{k+1} \underbrace{T(1)}_{=b} + cd\left(\frac{3^{k+1}-2d}{d}\right) \qquad (\because 2^k = d) \\
&= 3^{k+1} b + c\left(3^{k+1} - 2d\right) \\
&= 3^{k+1} b + c\left(3^{k+1} - 2d\right) \qquad \text{(where } T(1) = b \text{ is a constant)} \\
&\leq 3^{k+1}(b+c) = 3^k \times \underbrace{3(b+c)}_{\text{constant}} = O(3^k) \\
&= O(3^{\log_2 d}) \qquad (\because k = \log_2 d) \\
&= O(3^{\log_3 d \times \log_2 3}) = O\left((3^{\log_3 d})^{\log_2 3}\right) = O(d^{\log_2 3}).
\end{aligned}$$

5. Let $u_1 u_2 \ldots u_d$ be a binary string representing a non-negative integer $u \in \mathbb{Z}_{\geq 0}$. Describe an algorithm that computes u by reading the bits from left to right.

Ans: 5

Any binary number u can be computed by processing its bits $u_1 u_2 \ldots u_d$ from the most-significant to the least-significant bit order using this (Horner's rule) equivalence: $u = 2\left(2(\cdots 2(2u_1 + u_2) + u_3)\cdots\right) + u_d$.

So, the pseudocode to process the number from most to least significant bit order would read something like this:

```
1  /* Input: binary string u_1 u_2 ... u_d */
2  u ← 0
3  loop i from 1 to d:
4    u ← 2u + u_i
5  return u
```

6. In the lecture, you learnt an algorithm for modular exponentiation $a^b \bmod n$ using repeated squaring, which processes the binary representation of the exponent b from the least significant bit to the most significant bit order. Linking to the above question, is it possible to perform a similar approach for modular exponentiation while instead processing the bits from the most significant bit to the least significant bit order? If yes, write its pseudocode.

Ans: 6

The pseudocode below performs modular exponentiation by processing the bits of the exponent, say $b = (b_1 b_2 \ldots b_d)_2$, from the most significant bit to the least significant bit order.

Suppose that after processing the first $i-1$ bits, $b'_{\text{prev}} = (b_1 b_2 \ldots b_{i-1})_2$, the result of processing (say) r correctly holds $r = a^{b'_{\text{prev}}} \bmod n$. To expand the processing to include the next bit b_i and form $b'_{\text{curr}} = (b_1 b_2 \ldots b_{i-1} b_i)_2$, we have seen in the pseudocode above (refer previous question) that $b'_{\text{curr}} = 2b'_{\text{prev}} + b_i$.

Since this is the arithmetic required to update b' in each iteration when processing the bits from most to least significant order, the value of r must therefore be updated as $r = a^{b'_{\text{curr}}} \bmod n = a^{2b'_{\text{prev}}+b_i} \bmod n$.

This can be rearranged using laws of exponents as $r = \left(a^{b'_{\text{prev}}}\right)^2 \times a^{b_i} \bmod n$. When $b_i = 1$, this will be $r = \left(a^{b'_{\text{prev}}}\right)^2 \times a \bmod n$. Otherwise when $b_i = 0$, this will be $r = \left(a^{b'_{\text{prev}}}\right)^2 \bmod n$.

Thus, in each iteration, squaring the (prev) result r is common, and then, when $b_i = 1$, multiply by a, all modulo n. This is the logic applied in the pseudocode below:

Pseudocode:

```
1  /* Input: integers a ∈ ℤ, b = (b_1 b_2 ... b_d)_2 ≥ 0, n > 0*/
2
3  r ← 1
4
5  loop i from 1 to d:
6    r ← (r × r) mod n
7    if b_i = 1:
8      r ← (r × a) mod n
9
10 return r   /*r at this stage should hold a^b mod n*/
```

An additional inductive proof of correctness of the algorithm in the pseudocode is given below, for those interested.

When $d = 1$, let $b = (b_1)_2$.

When $b_1 = 0$: The algorithm starts with $r = 1$ (line 3). r is squared resulting in $r^2 = 1$ in line 5, while lines 6-7 are skipped since $b_1 = 0$. Hence the algorithm returns $r = 1$ in this case. This is correct since $a^0 \equiv 1 \pmod{n}$.

When $b_1 = 1$: The algorithm starts with $r = 1$ (line 3). r is squared resulting in $r^2 = 1$ in line 5, and lines 6-7 get executed and (the squared) r gets multiplied by a, giving $r = a \bmod n$. This is also correct since $a^1 \equiv a \pmod{n}$. Hence the base case when $d = 1$ holds.

Now assume that after processing the first $d-1$ bits, the algorithm correctly computed $r \equiv a^{b'} \pmod{n}$, where $b' = (b_1 b_2 \dots b_{d-1})_2$. Noow consider the next bit b_d. By Horner's rule, $b = 2b' + b_d$.

Checking the pseudocode, line 5 squares $r \equiv a^{b'} \pmod{n}$, yielding effectively $r^2 \equiv a^{2b'} \pmod{n}$.

When $b_d = 0$: Lines 6-7 are therefore skipped, so the exponent remains $2b' = 2b' + \underbrace{b_d}_{=0} = b$, hence $r \equiv a^{2b'+b_d} \equiv a^b \pmod{n}$ remains correct.

When $b_d = 1$: Lines 6-7 multiplies by a, giving $r \leftarrow r^2 \times a \equiv a^{2b'} \times a = a^{2b'+1} = a^b \pmod{n}$, because the exponent $2b' + \underbrace{b_d}_{=1} = b$.

Therefore, this correctly computes $r = a^b \bmod n$ for any $b \geq 0$.

7. Compute $7^{330} \bmod 13$ by hand using the repeated squaring method.

Ans: 7

Binary representation: $330 = (\overset{8\ 7\ 6\ 5\ 4\ 3\ 2\ 1\ 0}{1\ 0\ 1\ 0\ 0\ 1\ 0\ 1\ 0})_2 = 2^8 + 2^6 + 2^3 + 2^1$

Hence,

$$7^{330} \equiv 7^{256} \times 7^{64} \times 7^8 \times 7^2 \pmod{13}$$

i	$7^{2^i} \bmod 13$ (repeated squaring)	**result**
0	$7^{2^0} \bmod 13 = 7$	1
1	$7^{2^1} \bmod 13 = 49 \bmod 13 = 10$	$1 \times 10 \bmod 13 = 10$
2	$7^{2^2} \bmod 13 = 10^2 \bmod 13 = 100 \bmod 13 = 9$	10
3	$7^{2^3} \bmod 13 = 9^2 \bmod 13 = 81 \bmod 13 = 3$	$10 \times 3 \bmod 13 = 4$
4	$7^{2^4} \bmod 13 = 3^2 \bmod 13 = 9$	4
5	$7^{2^5} \bmod 13 = 9^2 \bmod 13 = 3$	4
6	$7^{2^6} \bmod 13 = 3^2 \bmod 13 = 9$	$4 \times 9 \bmod 13 = 10$
7	$7^{2^7} \bmod 13 = 9^2 \bmod 13 = 3$	3
8	$7^{2^8} \bmod 13 = 3^2 \bmod 13 = 9$	$10 \times 9 \bmod 13 = 12$
		$7^{330} \equiv 12 \pmod{13}$

8. Let n be an odd integer. For the bases $a = 1$ and $a = n - 1$, show that $a^{n-1} \equiv 1 \bmod n$.

Ans: 8

If n be an odd integer, then $n - 1$ is even.

- For $a = 1$: $1^{n-1} = 1 \equiv 1 \pmod{n}$. This is straight forward.
- For $a = n-1$: There are alternative ways to show that $(n-1)^{n-1} \equiv 1 \pmod{n}$ when n is odd. While a concise proof follows immediately from the congruence $n - 1 \equiv -1 \pmod{n}$, a more explicit proof is based on the binomial theorem.

$$\begin{aligned}(n-1)^{n-1} &= {}^{n-1}C_0\, n^0(-1)^{n-1} + {}^{n-1}C_1\, n(-1)^{n-2} + \cdots + {}^{n-1}C_k\, n^k(-1)^{n-1-k} + \cdots + {}^{n-1}C_{n-1}\, n^{n-1} \\ &= 1 - {}^{n-1}C_1\, n + {}^{n-1}C_2\, n^2 - {}^{n-1}C_3\, n^3 + \cdots - {}^{n-1}C_{n-2}\, n^{n-2} + {}^{n-1}C_{n-1}\, n^{n-1} \\ &= 1 - n\left({}^{n-1}C_1 - {}^{n-1}C_2\, n + {}^{n-1}C_3\, n^2 - \cdots + {}^{n-1}C_{n-2}\, n^{n-3} - {}^{n-1}C_{n-1}\, n^{n-2}\right)\end{aligned}$$

Notice that all terms on the r.h.s except the first term (=1) are multiples of n. Therefore, $(n-1)^{n-1} \equiv 1 \pmod{n}$.

9. Show that any integer $z > 0$ can be uniquely written in the form $z = 2^s t$, where $s \geq 0$ and t is odd.

Ans: 9

Consider the (minimal) binary representation of $z > 0$, say $z = (z_1 z_2 \ldots z_d)_2$. We know that $d \geq 1$.

Let $s \geq 0$ be the number of trailing zeros in the binary representation of z. In binary, each multiplication by 2 corresponds to a left shift, so the number of trailing zeros (s) equals the *largest* power of 2 dividing z.

Define $t = \frac{z}{2^s}$. In binary, t is obtained by removing the last s bits from the binary representation of z. Therefore, $t = (z_1 z_2 \ldots z_{d-s})_2$. Since s is the largest power of 2 dividing z, the least significant bit of t must be 1 and hence odd; otherwise, t would still be divisible by 2, contradicting the definition of s. Thus, $z = 2^s t$, where $s \geq 0$ and t is odd.

Uniqueness of s and t values can be posited on the uniqueness of the binary representation of z.

The number of trailing zeros (s) in z's representation is unique. The remaining $d - s$ bits of z that define $t = \frac{z}{2^s}$ is also unique. Therefore both s and t are unique.

10. Let $y^2 \equiv 1 \pmod{n}$. Determine solutions for $y \pmod{n}$ when n is prime.

Ans: 10

$y^2 \equiv 1 \pmod{n} \implies y^2 - 1 \equiv 0 \pmod{n} \implies (y-1)(y+1) \equiv 0 \pmod{n}$.

This means that the product $(y-1)(y+1)$ is divisible by n.

When n is a prime and divides a product, then it must divide at least one of the terms in the product. Therefore, $y \equiv 1 \pmod{n}$ or $y \equiv -1 \pmod{n}$ are its trivial square roots. Intuitively, this is true because primes cannot be factorized any further. For composite numbers, the factors of n may be distributed between $(y-1)$ and $(y+1)$, so additional solutions where $y \not\equiv \pm 1 \pmod{n}$ can arise.

This underpins observation 2 used in Miller-Rabin primality testing. In the algorithm, one computes repeated squares of the form $x_i = a^{2^i t} \pmod{n}$. If a non-trivial square root of 1 is encountered (that is, a value $x_{i-1} \not\equiv \pm 1 \pmod{n}$ such that $x_{i-1}^2 = x_i \equiv 1 \pmod{n}$), then n must be composite. Otherwise, n passes the test for that choice of base a and is probably prime.

11. Work out all the steps corresponding to one iteration of the Miller-Rabin primality test on $n = 21$ using the base (say) $a = 13$.

Ans: 11

$n = 21$, so $n - 1 = 20 = 2^2 \times 5$, hence $s = 2$ and $t = 5$.

$a = 13$. Denote $x_i = a^{2^i \times t} \bmod 21$.

$x_0 = 13^5 \bmod 21 = 13$ (which is computed using modular exponentiation, note).

Next, squaring previous, $x_1 = 13^2 \bmod 21 = 1$.

Since the first occurrence of 1 is at $i = 1$ we can conclude that the Fermat's little theorem test ($a^{n-1} \equiv 1 \bmod n$) will pass for the base $a = 13$. However, Miller Rabin performs the additional square root test. Check the previous value $x_0 = 13 \not\equiv \pm 1 \pmod{21}$. Therefore, since this is a non-trivial square root, MR will return 21 is definitely composite and terminate.

-=0=-

END

-=0=-